MDAF-based Explainable Failure Prediction for Network Functions in 5G/6G

Sukhyun Nam[1], Wonseok Choi[2], and James Won-Ki Hong[2]
[1]*POSTECH Institute of Artificial Intelligence, POSTECH, Pohang, Korea*
[2]*Department of Computer Science and Engineering, POSTECH, Pohang, Korea*
{obiwan96, ws4583, jwkhong}@postech.ac.kr

NOMS 2026-2026 IEEE Network Operations and Management Symposium | 979-8-3315-9268-4/26/$31.00 ©2026 IEEE | DOI: 10.1109/NOMS69089.2026.11668370

***Abstract*—The evolution toward 6G networks introduces unprecedented complexity, requiring proactive and explainable management mechanisms. This paper presents an implementation of the Management Data Analytics Function (MDAF) as defined by 3GPP, focusing on failure prediction and explainable fault analysis for 5G network functions (NFs). The proposed framework integrates (i) multi-domain (i.e., architectural-domain) metric and log collection across RAN, Core, and Application domains, (ii) time-series deep learning models (CNV-GRU with focal loss and soft labeling) for predictive analytics, and (iii) a hybrid explainability layer combining Shapley Additive exPlanations (SHAP) -based feature attribution and LLM-driven natural language reasoning. Using a Kubernetes-deployed OpenAirInterface testbed, the system achieved up to 78.6% F1 score for detection and 73.0% for 3-minute-ahead prediction, demonstrating the feasibility of AI-assisted cross-domain OAM (Operation, Administration, and Maintenance). Moreover, the integration of SHAP and LLMs enabled interpretable causal explanations that identified the responsible NF with 77.8% accuracy, providing actionable insights for self-healing management. This study demonstrates the feasibility of implementing MDAF as a practical AI-driven OAM entity and offers a foundation for future autonomous and 6G network management.**

***Index Terms*—MDAF, 5G/6G, Failure Prediction, Explainable AI, LLM**

I. Introduction

The evolution of mobile networks from 4G to 6G has brought about an unprecedented surge in traffic volume, service heterogeneity, and management challenges. Contemporary mobile operators are facing the dual challenge of scaling network capacity while minimizing capital (CAPEX) and operational (OPEX) expenditures. To address these issues, Network Function Virtualization (NFV) has been adopted, enhancing flexibility and resource efficiency but also introducing new challenges in observability, reliability, and fault isolation.

In particular, 5G core networks, consisting of decomposed service-based functions such as Access and Mobility Function (AMF), Session Management Function (SMF), and User Plane Function (UPF), exhibit tight interdependencies across distributed domains. A failure in a single Network Function (NF) instance can propagate to other domains which can trigger service degradation or even network-wide outages. Consequently, proactive failure prediction and self-healing management have become central goals of next-generation OAM (Operations, Administration, and Maintenance) systems. The 3GPP[1] standardization body has therefore proposed analytic and intelligent control entities—namely the Network Data Analytics Function (NWDAF, [1]) and the Management Data Analytics Function (MDAF, [2])—to enable AI-assisted, data-driven network management. In addition, O-RAN Alliance proposed the Intelligent Radio Access Network (RAN) Intelligent Controller (RIC, [3]) to optimize RAN control.

Among these analytic entities, MDAF represents a higher-level analytics layer that aggregates data from multiple NWDAFs and other management functions to perform end-to-end (E2E) inference across the RAN, Core, and Application domains. First introduced in 3GPP Release 16, MDAF aims to provide unified E2E visibility and root-cause analysis capabilities that cannot be achieved by isolated domain-level analytics.

Among the use cases proposed for MDAF by 3GPP is fault management [4]. This includes scenarios such as predicting failures or performance degradation in NF, performing root cause analysis, and generating recommended actions to support network and service management. Importantly, faults in a given network function may not stem from that function alone. Possible causes include overload in the local node, failures in other co-located functions, erroneous connection requests, or database errors in other network functions—demonstrating that failures can originate from any part of the network domain. However, existing network failure prediction systems primarily focus on improving predictive accuracy, often providing results that are difficult for operators to interpret.

Therefore, data collection strategies for network status monitoring must support cross-domain contextual analysis, as enabled by MDAF. By leveraging MDAF to predict failures in NFs, it becomes possible to uncover failure causes that lie not only within the target function but also in other NFs or even different domains. This cross-domain reasoning capability is made possible by recent advances in Artificial Intelligence (AI), particularly the emergence of LLMs (Large Language Models) with inference capabilities.

In this regard, explainable AI (XAI) plays a vital role. However, XAI remains an underexplored area within the research domains of network anomaly detection and failure prediction. By integrating XAI into the MDAF framework, we

[1]Global telecom standards organization for 4G/5G technologies, https://www.3gpp.org/

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:51:38 UTC from IEEE Xplore. Restrictions apply.

enable not only accurate failure forecasting but also human-understandable reasoning that supports root cause analysis and actionable insights.

This work focuses on one essential function of MDAF—failure prediction and explainable failure management for NFs in 5G/6G environments. Our objective is not merely to detect anomalies. We aim to forecast potential NF failures and to provide interpretable explanations that assist operators in proactive maintenance. To achieve this, we integrate (i) multi-domain (i.e., architectural-domain) metric collection compliant with 3GPP OAM standards, (ii) time-series deep learning models for predictive analytics, and (iii) explainability mechanisms combining Shapley Additive exPlanations (SHAP)-based feature attribution with LLM-driven natural language reasoning for explainable failure prediction.

II. Background and Related Work

A. Management Data Analytics Function (MDAF)

3GPP defines a layered management and analytics stack where domain-level analytics (NWDAF) are complemented by the MDAF for cross-domain, E2E reasoning. The measurement and KPI foundations are specified in TS 28.552 [5] (performance measurements) and TS 28.554 [6] (KPI modeling), while TS 28.104/28.533 [2], [4] describe management architecture, reference points, and orchestration aspects relevant to deploying analytics functions within operational management systems. Collectively, these specifications argue that (i) analytics must correlate RAN–Core–Application signals; (ii) E2E KPIs must be derived from multi-domain measurements; and (iii) analytics outputs should be consumable by higher-level assurance and control loops (e.g., policy, self-healing).

MDAF has emerged as a key functional entity in the 3GPP management architecture to enable cross-domain analytics and E2E network intelligence. According to 3GPP TS, MDAF is responsible for consolidating analytics from RAN, Core, and Service domains, thereby enabling higher-level reasoning (service-level KPIs, SLA assurance, convergence-based optimization, energy saving, mobility assist, and E2E fault management [4]).

To the best of our knowledge, this paper presents one of the first implementations that operationalizes the MDAF concept. While prior work [7]–[9] discusses MDAF mainly at a conceptual level, our study operationalizes it by integrating real-time multi-domain data collection, time-series deep learning-based failure prediction, and explainable reasoning. By combining SHAP-based attribution and LLM explanations, the framework goes beyond numerical prediction to deliver interpretable and actionable insights.

B. LLM-based Explainable Anomaly Detection

Large Language Models (LLMs) are deep neural network-based models used for a wide range of natural language processing (NLP) tasks. LLMs are trained on vast amounts of textual data and are capable of understanding context, answering natural language queries, and generating coherent text. Recent LLMs such as GPT [10], Gemma [11], and DeepSeek [12] leverage billions of parameters to learn contextual relationships between words.

Recent studies, such as LLMeLog [13] and LogiCode [14] explored the use of LLMs for anomaly detection by translating system logs into semantically enriched representations, yet none address multi-domain failure prediction. However, research on explainable network anomaly detection is still in its early stages. Most of them are confined to general intrusion datasets and device-level telemetry, rather than end-to-end service-oriented NFs.

HuntGPT integrates traditional anomaly detectors with LLMs to generate natural-language rationales and triage guidance for security analysts. The pipeline uses model outputs (scores, salient features) and relevant logs as prompts so the LLM can hypothesize causes, and suggest follow-ups. They conducted an anomaly detection experiment on the NSL-KDD dataset [15] based on Random Forest model and build an explain module based on LLM. M.K. Hooshmad *et. al.* [16] also conducted detection experiments on the NSL-KDD and UNSW-NB15 [17] datasets with an ensemble classification system based on XGBoost. In particular, they proposed several techniques for handling data imbalance and provided an explanation of the prediction results of the model through SHAP.

While prior studies have primarily focused on intrusion detection based on dataset, our work goes a step further by providing interpretable insights into the root causes of failures, thereby supporting operators in making more informed decisions. The explainability offered by the combined use of SHAP and LLM extends beyond predictive accuracy, serving as a critical technical contribution for practical decision-making in real-world network operations.

III. Design and Implementation

A. System Overview

In this paper, the term "multi-domain" follows the 3GPP OAM/MDAF terminology and refers to different architectural or functional domains, such as the RAN, Core Network, and Application/Service segments, within a single operator network. We do not assume cross-organizational or cross-administrative data sharing. Instead, all domains considered in this work are under common or coordinated administrative control, allowing unified data collection for analytics purposes. The proposed MDAF-based explainable failure prediction framework consists of three main modules (Fig. 1):

1) MDAF-Collector: Gathers real-time metrics and logs from RAN, Core, and Application domains, storing them in a time-series database after pre-processing.
2) Failure Prediction Module: Trains time-series deep learning models to forecast future NF failures.
3) Root Cause Analysis Module: Interprets SHAP-based feature importance and invokes an LLM to produce human-readable root-cause explanations and corrective recommendations.

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:51:38 UTC from IEEE Xplore. Restrictions apply.

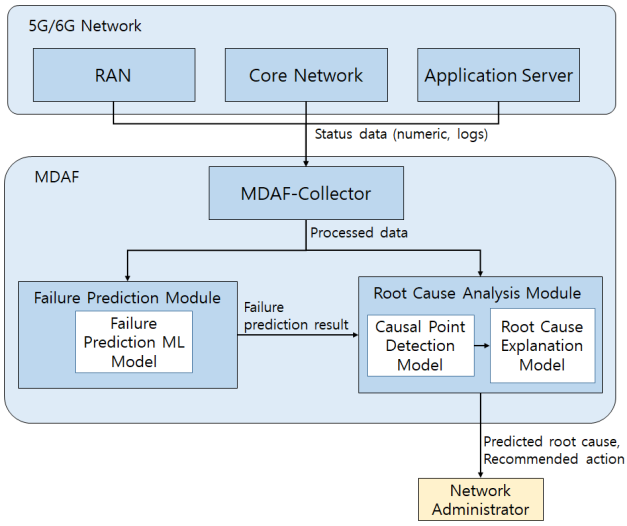


Fig. 1. MDAF-based Explainable Network Function Failure Prediction System Structure

B. MDAF-Collector

In line with 3GPP TS 28.552 (Performance Measurements) [5] and TS 28.554 (E2E KPIs) [6], we identified key metrics spanning RAN, Core, and Application domains. The measurements and their operational significance are summarized in Table I.

The proposed MDAF assumes that NFs are operated within a virtualized environment, where each NF is deployed as a separate VM or Pod (in a Kubernetes[2]-based setup). Accordingly, metrics such as CPU usage, memory consumption, network traffic volume, and disk usage were collected per-Pod using Prometheus [18] for all domains in a Kubernetes environment, with adjustable granularity (per-Pod vs domain-level aggregation).

In addition to these readily available system metrics, we propose collecting additional domain-specific context metrics that are not directly supported by default monitoring systems. Since 3GPP TS documents do not specify concrete metrics for MDAF, we define custom metrics based on TS guidelines to indicate overload status in each domain. Given that our experimental setup is based on a simplified 5G environment, overload conditions are primarily assessed based on the connection status of User Equipments (UEs) and Protocol Data Unit (PDU) sessions.

For the RAN domain, we parse Radio Resource Control (RRC)-related logs from the next generation Node B (gNodeB, gNB) to monitor the state transitions of RRC connections within a recent time window (**RRC State Count**). In addition, although collecting such log data may be challenging in real networks due to privacy and security concerns, the 3GPP TS does suggest the possibility of collecting data at both the UE and application levels. To supplement this, we also parse logs from the UE to count the number of failed session connection attempts (**UE Fail State Count**).

Furthermore, in order to assess the E2E service quality of the network, we introduced a dedicated UE, termed the Service Level Objective (SLO) Violation Check UE, which periodically generates traffic toward an application server through the 5G environment. This UE measures delay and counts the number of failures in requests to detect SLO violations. It records the frequency of SLO violations within a recent time window (**SLO Violation Count**).

In the Core domain, metrics are primarily collected in relation to session management. Specifically, we correlate and analyze logs from the AMF and SMF: when a PDU session creation request is received by the AMF, measure the elapsed time until the SMF completes the PDU session establishment (**PDU Session Delay**). Similar to the RAN domain, we measured the ratio between the number of newly registered connections and the number of terminated connections in the AMF logs. (**AMF Registration Rate**).

To assess potential network traffic overload, we calculate the ratio of transmitted to received traffic at the UPF over a defined time interval (**UPF Userplane Throughput**). Additionally, we parse SMF logs to count the number of dropped sessions within a recent period (**SMF Session Drop Count**).

We also monitored the frequency of error-level log messages from each NF. During our experiments, whenever an [error]-level log entry was observed from a NF, we monitored whether the network continued to operate normally afterward. If the system remained stable, the corresponding error message was added to a known error list, and the occurrence frequency of such known errors was tracked over time. In contrast, if an error was followed by an abnormal network state (e.g., a sudden halt in traffic flow), the corresponding time point was tagged as a failure prediction target rather than a known error. Among the known errors included in our dataset, there are ten representative failure cases. For example, the UDR shows

[2]An open-source cloud-native container orchestration platform. Official website: https://kubernetes.io/

TABLE I
METRICS COLLECTED IN MDAF FRAMEWORK

Metric	Description	Domain
CPU Usage	CPU usage mean (by Pod)	All
Mem Usage	Memory usage bytes mean (by Pod)	All
Network Transmit	Network transmit bytes mean (by Pod)	All
Disk Write	Disk write bytes mean (by Pod)	All
RRC State Count	Count RRC status change in gNB log (CONNECTED/IDLE, etc.)	RAN
UE Fail State Count	Count of UE connection failures	RAN
SLO Violation Count	SLA violations detected at UE side	RAN
PDU Session Delay	Latency between PDU session request and accept	Core
AMF Registration Rate	Success ratio of AMF registration	Core
UPF Userplane Throughput	UPF GTP-U throughput ratio	Core
SMF Session Drop Count	Number of dropped SMF sessions	Core
Known Error Count	Non-critical error occurrences	RAN, Core

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:51:38 UTC from IEEE Xplore. Restrictions apply.

"*No data available for AccessAndMobilitySubscriptionData*" when it fails to respond to subscription data requests from the UDM or AMF, and the AMF shows "*Could not find Nsmf_PDUSession URI*" when it fails to retrieve the SMF service URI from the NRF. These errors are typically caused by temporary database failures due to overload and often lead to session creation and management failures—early indicators of impending network faults.

We also considered collecting application-layer metrics (also suggested in 3GPP TS); however, since application-level behavior is highly dependent on the specific services deployed on the server, we concluded that only universally applicable metrics—such as resource utilization—should be collected at the application layer. The transport domain was not considered in our virtualized 5G network.

The proposed metrics can be collected in real time. However, how these metrics are utilized within MDAF can be flexibly configured. As previously mentioned, resource usage data can be stored per Pod, per domain, or per node (i.e., the node on which each Pod is running). But as will be discussed in the causal analysis section, in our study, since one of the primary objectives was to identify the specific NF responsible for a failure, we opted to use per-Pod data without aggregation.

Furthermore, for failure prediction, the change in collected data is often more informative than the raw value itself (Raw). Therefore, instead of using raw values, we also considered the difference (Diff) between consecutive data points, as well as the variance (Var) computed over a sliding window to capture short-term fluctuations. In our experiments, we compared the failure prediction performance when using Raw, Diff, and Var representations of the metrics.

In addition, machine learning models generally require input features to be on a similar scale. To address this, we analyzed the skewness of each metric—a statistical measure of the asymmetry of the data distribution. For metrics exhibiting high positive skewness, we applied a logarithmic transformation to reduce distributional bias. In our experimental results, both the AMF Registration Rate and UPF Userplane Throughput showed skewness values exceeding 20, prompting us to apply the log transformation. Finally, before feeding the metrics into the failure prediction model, we applied normalization to standardize each metric to have a zero mean and unit variance. Through this process, the MDAF-Collector stores state data in a normalized format that enables accurate and stable failure prediction.

C. Failure Prediction Module

Following the MDAF-Collector, the Failure Prediction Module receives data from it and predicts whether a failure will occur after a certain time interval. It leverages a machine learning model that has been trained on a pre-collected dataset to determine the likelihood of future failures. The dataset consists of time-series metrics and the corresponding timestamps of failure and recovery events. To utilize this dataset for training, input–output pairs must be constructed appropriately. Specifically, we employ a sliding window approach to form

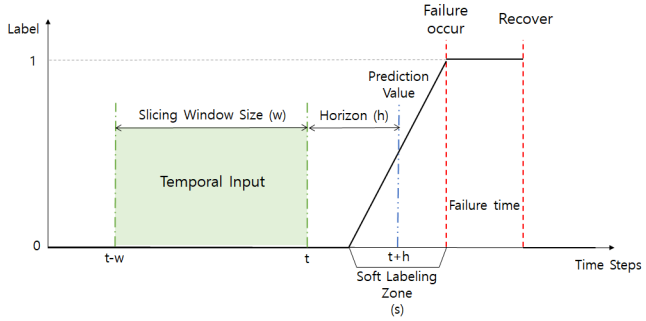


Fig. 2. Soft labeling for the temporal prediction models. X-axis represents the temporal dimension (*t-w* to *t+h*), while the Y-axis denotes the target variable (y).

temporal input sequences, and label each sample as 1 if a failure occurs within a fixed horizon after the window, and 0 otherwise. In addition, to better capture the gradual degradation before a failure, we introduce a soft labeling window, within which training samples are labeled with values between 0 and 1 based on their proximity to the failure time.

Fig.2 illustrates an example of defining input intervals and prediction targets using linear soft labeling. Green dashed lines represent the observation window (input sequence), and blue dashed lines denote the forecasting horizon (predicted values). A sliding window with a fixed size w (window size) moves over the time-series data, allowing the collected metrics within each window to be used as model inputs. The label is determined based on whether a failure occurs after a fixed time interval h (horizon) from the end of the window.

In the soft labeling zone preceding the failure, labels decrease progressively with the distance from the failure point, instead of using hard binary labeling. Since we adopt a linear labeling scheme, with s as the soft labeling size, the label value at a point $dist$ time units away from the failure is given by: $y = 1 - \frac{dist}{s}$. This enables the model to learn temporal transitions more effectively, particularly in situations where anomalies typically evolve progressively rather than instantaneously.

If the prediction target time falls within the actual failure period (i.e., between the failure onset and recovery time), the label y is set to 1. However, in the context of failure prediction, the prediction system is assumed to be inactive once a failure has already occurred. Therefore, when the sliding window reaches the failure period, those data points are excluded from training and evaluation. An exception is made when the horizon is set to zero. In this case, the task becomes failure detection rather than prediction, and thus, such instances are included in the dataset.

Using this labeling strategy, the input data at time t and the corresponding prediction target are defined as follows. The input features have (window $\times$ features) dimension. For each time step, we generated input–output pairs (X, y), which were then used to construct the training dataset for the prediction

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:51:38 UTC from IEEE Xplore. Restrictions apply.

model.

$$X(t) = X_{t-w:t} \in \mathbb{R}^{w \times d}$$

$$y(t) = \begin{cases} 1, & \text{if within failure period,} \\ 1 - \dfrac{dist}{s}, & \text{if } 0 \leq dist \leq s, \\ 0, & \text{if } dist > s. \end{cases}$$

We explored several neural architectures optimized for temporal modeling, including the Long Short-Term Memory (LSTM, [19]) network and the Gated Recurrent Unit (GRU, [20]).

- LSTM: Baseline sequence model capturing long-term dependencies.
- GRU: Lightweight variant reducing vanishing gradient issues.
- GRU-Attn: Augments GRU with a temporal attention mechanism [21] to emphasize critical patterns preceding failures.
- CNV-GRU: Incorporates 1-D convolutional filters prior to GRU to capture short-term fluctuations.

The collected data is time-series in nature, and detecting anomalies within the temporal progression of the data is crucial. Therefore, we ensured that the temporal order of the data was preserved during both training and testing.

In the case of failure prediction experiments, actual failure events are relatively rare, resulting in a significant imbalance where normal samples far outnumber failure-labeled samples. To address class imbalance, we adopted a 2-fold evaluation strategy, splitting the data chronologically for training and testing. In addition, we adopted the Focal Loss (Lin *et al.*, [22])), which extends the standard cross-entropy loss. All hyperparameters (hidden size, number of layers, dropout, α, etc.) are optimized using Optuna [23], a framework for automated hyperparameter optimization.

D. Root Cause Analysis Module

The XAI module integrates SHAP-based local interpretability with LLM-driven reasoning. Explainability is provided through a two-stage process:

1) **Causal Point Detection**: Using SHAP, each feature's contribution to the failure probability is computed, to identify the NF most likely responsible for the failure.
2) **Root Cause Explanation**: The top contributing features and associated log snippets are formatted as textual prompts and passed to an LLM. The LLM generates human-readable explanations in the format.

SHapley Additive exPlanations (SHAP) [24] helps to understand the predictions of complex machine learning models. The SHAP formula averages the contribution of each feature to the model prediction across all possible subsets of features. This allows us to assess the influence of individual features in the predicted failure, enabling the identification of the NF responsible for the failure. For example, if the CPU and memory usage of the AMF Pod and the AMF Registration Rate show a high influence, it can be determined that AMF is the cause of the failure. We pre-assigned associated NFs to each context feature (e.g., the influence of UE failure counts on gNB, AMF, and the Authentication Server Function, AUSF).

After identifying the most influential features via SHAP, we format the top-k feature names, their contributions, and related log snippets into a natural-language prompt and feed it to the LLM. The LLM then produces a concise explanation describing the likely root cause and a suggested corrective action, e.g., '*AMF overload due to excessive registration requests. Scale out oai-amf pod.*' This process transforms raw analytic output into human-interpretable operational insight.

Since our research was not focused on how to utilize LLMs, we selected three models with approximately 20 parameters each from the most commonly used LLMs in the Ollama [25] API: GPT-OSS:20B [10], Gemma3:27B [11], and DeepSeek-r1-Llma3:14B [12]. For the DeepSeek model, we used the distilled version, which was trained through Llama3 [26].

E. Implementation Environment

To build a 5G testbed in a virtual environment, we established a container-based networking environment using Kubernetes and deployed the 5G testbed. Fig. 3 illustrates the detailed architecture of the testbed. In order to make the RAN, Core, and Application domains resemble a real-world 5G setup, they were configured on separate nodes, and the testbed is structured as follows:

- **5G Core (OAI [27])**: Core network functions components deployed as Pods.
- **RAN (UERANSIM [28])**: gNodeB and UE emulators generating traffic and control procedures.
- **Application Server (NGINX [29])**: Web server providing end-user services.
- **MDAF-Collector**: Prometheus [18] (metrics), Loki [30] (logs) and Grafana [31] (visualization)
- **MDAF-AI-Analyzer)**: External server running InfluxDB [32] and Python-based ML engine.

To induce failures in network functions while maintaining a traffic environment similar to real-world networks, we applied intentional overload conditions. Specifically, we repeatedly generated and removed a random number of UEs approximately every 10 seconds to create excessive connection and disconnection requests. The generated UEs were configured to automatically send both background and overload traffic to the web application server using K6 and H2Load [3].

As described in a Subsection III-B, a dedicated SLO Violation Check UE was continuously operated to monitor service quality. The SLO thresholds were defined based on the average response latency and failure rate observed in the experimental environment, and SLO violations were checked every 10 seconds by this UE. The *MDAF-Collector* collects data by communicating with the Prometheus/Loki API. All code for collecting the proposed metrics was implemented using rule-based parsing and stored in a Influx DB database.

[3]HTTP/2 performance benchmarking, load testing tools. https://k6.io/, https://nghttp2.org/documentation/h2load.1.html

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:51:38 UTC from IEEE Xplore. Restrictions apply.

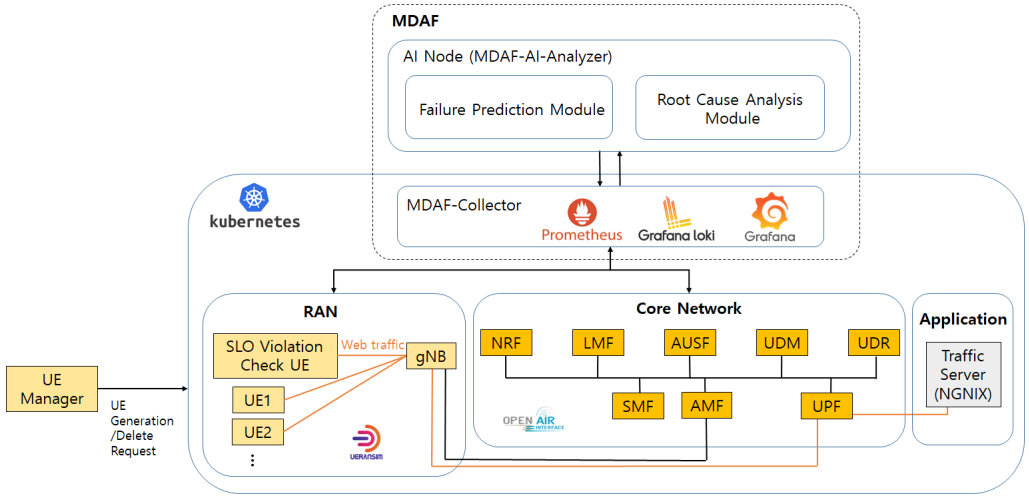


Fig. 3. MDAF-based Network Function Failure Prediction Experiment Testbed

The corresponding implementation is available on our GitHub repository[4]. The *MDAF AI Node* runs outside the cluster and communicates with the MDAF-Collector. The AI nodes consist of two GPU nodes (RTX 5000 and A6000), where one runs the failure prediction model, and the other runs the LLM model.

IV. Results and Evaluation

A. Data Collection

In our experimental environment, we generated network traffic over a period of 10 days while monitoring for failure events. Since background traffic from UEs was continuously present in our scenario, we specifically observed cases where network traffic suddenly dropped, which could indicate a failure. In such cases, we examined the logs of each NF to identify the faulty NF and to record the corresponding failure time. Pod self-restarts were also treated as failure events. Additionally, we monitored the system to determine when traffic had returned to normal and recorded the recovery time accordingly.

A total of 13 failures were observed during the experiment. As shown in the Table II, failures were distributed across various NFs, and in one case, the failure was not caused by a specific NF but by an overload in the Kubernetes platform itself, which led to a system-wide halt. We then evaluated whether these failures could be effectively predicted. Also, all experimental results other than performance measurements according to window and horizon were conducted assuming a window of 30 minutes and a horizon of 6 minutes, and using the Diff method.

[4]https://github.com/obiwan96/FP_with_LLM

TABLE II
Distribution of Failure Events by NF and Type (Total 13 cases)

NF	Restart	Complete Failure	Total
gNB	3	2	5
SMF	1	2	3
AMF	1	1	2
UDR	1	1	2
Kubernetes (node)	0	1	1
Total	6	7	13

After collecting the data, we analyzed whether each feature exhibited a statistically significant correlation with the labeling outcome. We computed the p-value between each feature and the corresponding label. In general, a p-value below 0.05 is considered statistically significant. While the average p-value across all 51 features was relatively high at 0.09, some features (memory usage, CPU usage, AMF Registration Rate, PDU Session Delay, RRC State Counts, and UPF Userplane Throughput) showed significant correlations ($p < 10^{-17}$).

These results suggest that the collected metrics are indeed meaningful for failure prediction. Furthermore, given that the data is time-series in nature, we hypothesized that even features with relatively low correlation might still contribute to anomaly detection through temporal variation. Moreover, this is particularly important for subsequent SHAP analysis, which aims to identify the causal point NF. Therefore, we chose not to exclude any features based on p-value alone, and used all 51 features—each represented by one value per time step—as input for the machine learning models.

B. Performance Comparison

We conducted failure prediction experiments using the collected data. Hyperparameters were optimized via Optuna; best results achieved with hidden size = 256, dropout = 0.3, etc.. The time interval between samples (step size) was set to 3 minutes, and both the horizon and window size parameters were varied in multiples of 3 minutes. As described in the subsection III-C, we adopted a 2-fold evaluation strategy, measuring the F1 score on the test set in each fold and reporting the average performance.

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:51:38 UTC from IEEE Xplore. Restrictions apply.

We experimented with all three types of input representations: Raw, Diff, and Var. The experimental results are summarized in the table III. When the horizon was set to 0 (i.e., in the detection setting), the best F1 score reached 78.6%. For failure prediction 3 minutes in advance, the performance was 73.0%, gradually decreasing as the horizon increased. Overall, the results indicate that the model performs best when the window size is set to 30 minutes, suggesting that this configuration allows the model to most effectively capture and understand the temporal patterns in the data. When the window size increases to 45 minutes, performance degrades, likely due to the excessive input length.

In addition, models using Raw input representations consistently exhibited significantly lower performance compared to the other two approaches. Among the alternatives, the Diff representation generally achieved better results, highlighting the importance of capturing short-term changes rather than relying on absolute values.

We also conducted an ablation study to evaluate the effectiveness of each step proposed to address the class imbalance problem in our failure prediction model. Specifically, we compared the performance of the model with and without each of the proposed components, as well as the overall performance across all model variants (Table IV).

While the performance across most models—excluding LSTM—was relatively similar, the CNV-GRU model, which incorporates convolutional layers optimized for extracting meaningful features from large-scale inputs, consistently outperformed the others. Furthermore, we can confirm that Focal Loss and soft labeling steps contribute to meaningful performance improvements.

TABLE III
F1 SCORES (%) BY WINDOW AND HORIZON UNDER DIFFERENT PREPROCESSING METHODS, CONV-GRU (RAW, DIFF, VAR).

Horizon (min)	Window (min)	Raw	Diff	Var
0	15	59.3	69.7	68.9
	30	72.1	**78.6**	76.7
	45	63.6	69.2	71.3
3	15	56.6	68.3	67.8
	30	63.6	72.3	**73.0**
	45	56.9	69.6	70.3
6	15	58.3	69.2	66.9
	30	57.8	**71.3**	71.1
	45	57.3	68.2	67.3
9	15	53.4	66.3	65.3
	30	59.7	**67.3**	67.1
	45	56.8	65.6	66.6
15	15	43.0	59.2	57.3
	30	48.1	**62.3**	61.9
	45	43.9	61.5	61.6

TABLE IV
ABLATION STUDY: IMPACT OF MODEL ARCHITECTURE AND FUNCTIONAL COMPONENTS (WINDOW: 30M, HORIZON: 6M, DIFF)

Configuration	F1 (%)	ΔF1 (%)
(A) Model Architecture Comparison		
LSTM (baseline)	64.3	-
GRU	65.5	+1.2
GRU-Att	67.6	+2.1
CNV-GRU	**68.4**	+0.8
(B) Functional Component Enhancement		
+ Focal Loss (on CNV-GRU)	69.5	+0.9
+ Soft Labeling	**71.3**	+1.8

TABLE V
CROSS-DOMAIN EFFECT AND IMPACT OF LOG-BASED METRICS IN MDAF FRAMEWORK (CNV-GRU, WINDOW: 30M, HORIZON: 6M, DIFF)

Domain Scope	Data Type	F1 (%)	ΔF1 (%)
RAN (single-domain)	Resource Only	63.7	–
Core (single-domain)	Resource Only	65.8	–
Multi-domain (Resource)	Resource Only	**66.3**	+2.6
RAN (single-domain)	Resource + Context	68.7	+5.0
Core (single-domain)	Resource + Context	69.1	+3.3
Multi-domain (MDAF)	**Resource + Context**	**71.3**	**+5.0**

C. Cross-Domain Effect

To validate the effectiveness of MDAF in practical settings, we conducted additional experiments by limiting the input data to a single domain or to only resource usage metrics, excluding the additional context metrics we collected from log analysis. In the single-domain evaluation, both the input data and the failure labels were restricted to the NFs belonging to the corresponding domain.

The results (Table V) showed a noticeable drop in performance when only a single domain was used, suggesting that cross-domain context—such as transient overloads in other domains or information from session management functions—plays a critical role in understanding system behavior and improving failure prediction accuracy. Moreover, within the same domain, we observed a substantial improvement in performance when contextual features were included. These findings confirm our hypothesis that collecting the MDAF-specific metrics contributes meaningfully to improving failure prediction performance.

D. Explainability Analysis

We also evaluated the performance of the Root Cause Analysis Module. This evaluation was conducted in two parts: (1) SHAP-based causal point analysis was performed only on the subset of failures correctly predicted by our Failure Prediction Module, and (2) we assumed that all failures were correctly predicted, and evaluated how accurately the LLM identified the actual root cause NF across all failure cases.

For the **SHAP-based analysis**, among the observed failures, one case caused by Kubernetes-level overload was excluded

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:51:38 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
EXPLAINABILITY PERFORMANCE OF SHAP AND LLM-BASED REASONING (MEAN OF 3 LLM MODELS)

Module	Evaluation Setting	Accuracy (%)
SHAP (Feature Attribution)	Causal NF Identification	77.8
LLM (Single Response)	Cause Reasoning (Mean)	69.5
LLM (Two Responses)	Cause Reasoning (Mean)	77.7

from the SHAP-based analysis, as it could not be attributed to any specific NF using feature-level explainability. The root cause analysis experiments were conducted using the best-performing model for each horizon value, based on prior failure prediction results. Across the four models tested, the SHAP-based analysis correctly identified the responsible NF in **77.8%** of cases on average. For instance, using the model with a window size of 30 minutes and a horizon of 6 minutes, the Failure Prediction Module correctly detected 9 failures, among which SHAP accurately identified the root cause NF in 7 cases. In misclassified instances, SHAP usually assigned high importance to memory usage features, and it was observed that the mistakenly identified NFs also exhibited high memory usage at the time—suggesting a potential source of confusion.

We further evaluated whether the **LLM could accurately identify the root cause**, assuming perfect failure prediction and causal point detection. We conducted experiments on 12 failure cases (excluding the one caused by Kubernetes overload). For each case, we provided the LLM with the logs (from the corresponding NF) generated within the 30 minutes prior to the failure—filtered to include only entries of warning level or higher—along with the SHAP results. We had previously recorded the root causes and solutions for each failure while operating the testbed, and we evaluated the LLM by checking manually the response include ground truth cause.

In preliminary tests, LLMs given unrestricted prompts produced multiple possible explanations, often including the correct root cause. In fact, over 80% of the unrestricted responses from all LLMs contained the correct root cause. However, to more rigorously assess their performance, we modified the prompt to explicitly instruct the LLMs to output only a single answer. Under this stricter setting, the correct response rates were as follows: GPT-OSS achieved **66.7%**, Gemma3 achieved 50%, and DeepSeek-r1-Llama3 achieved 41.7%. The experimental results revealed that the LLMs heavily relied on SHAP results, often attributing failures to high memory usage and recommending general scaling strategies. Interestingly, we explicitly instructed the LLMs to focus on the logs rather than the SHAP outputs; all three models achieved over 65% accuracy in identifying the correct root cause. Furthermore, when the models were allowed to generate up to two responses, their accuracy **exceeding 75%**.

Furthermore, the experiments revealed that modern LLMs possess a non-trivial degree of domain knowledge regarding NF behavior. For example, in cases involving session management failures (e.g., AMF or SMF-related errors), the models often provided accurate diagnoses based on log patterns and our provided metrics. These results demonstrate that, as intended, the LLMs were able to generate explainable and context-aware diagnostic insights for network failures. However, due to the limited number of failure samples and the lack of a well-defined metric for quantitatively evaluating the quality of LLM-generated root cause explanations, further research is needed to develop appropriate evaluation methodologies and to explore more effective ways of integrating LLMs into failure analysis pipelines.

V. CONCLUSION AND FUTURE WORK

This paper presented an MDAF-based explainable failure prediction framework for 5G/6G NFs. The system implements the 3GPP MDAF concept by unifying cross-domain data collection, time-series failure prediction, and explainable AI-based reasoning. We evaluated our proposed model on a Kubernetes-deployed OpenAirInterface testbed.

Although the testbed was constructed at the laboratory scale under unified administrative control of RAN, Core, and service domains—consistent with typical single-operator 5G deployments—the observed failures correspond to the most frequently reported faults in real-world networks. The framework relies solely on standard OAM metrics already available in operational systems and introduces no additional probing traffic or packet-level inspection. Its preprocessing and cross-domain correlation involve lightweight aggregation and alignment with computational complexity linear in the number of monitored NFs and metrics. Model training is performed offline, while online inference consists of lightweight forward passes. SHAP-based explanation and LLM inference are triggered only upon failure detection and selectively applied through a hierarchical analytics design to a limited subset of suspicious NFs. These characteristics ensure high scalability and make the proposed approach directly applicable to operational networks.

We will extend our research in three directions. First, we will expand it toward the design of a **closed-loop network management** architecture capable of automatically incorporating RCA results into operational control. Second, we will continue in-depth investigation of **causal analysis through multi-domain correlation modeling**. Finally, to better leverage LLMs, we will explore techniques such as **Retrieval-Augmented Generation (RAG) and fine-tuning** in order to adapt them more effectively to the target task.

The proposed framework indicates that MDAF has the potential to evolve from a conceptual management entity into an operationally deployable, explainable, and autonomous management function in future 6G networks.

ACKNOWLEDGMENT

This work was supported by Institute of Information & Communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (RS-2024-00392332, Development of 6G Network Integrated Intelligence Plane Technologies) and Samsung Electronics Co., Ltd.

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:51:38 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] 3GPP Technical Specification Group Services and System Aspects. 5g system; architecture enhancements for 5g system (5gs) to support network data analytics services. Technical Specification (TS) 23.288, 3GPP, 2019. Stage2.

[2] 3GPP Technical Specification Group Services and System Aspects. 5g; management data analytics function (mdaf); stage2. Technical Specification (TS) 28.533, 3GPP, 2020. Stage2.

[3] O-RAN Alliance. O-rannear-real-time ran intelligent controller (near-rtric) specification. Specification "First Release", O-RAN Alliance, 2022. xApps / rApps architecture.

[4] 3rd Generation Partnership Project (3GPP). Telecommunication management; Management and orchestration; Fault Management (FM); Concept and requirements. Technical Specification TS 28.104, 3GPP, 2023. Release 17.

[5] 3rd Generation Partnership Project (3GPP). Telecommunication management; Management and orchestration; 5G end to end Key Performance Indicators (KPIs); Performance measurements. Technical Specification TS 28.552, 3GPP, 2023. Release 17.

[6] 3rd Generation Partnership Project (3GPP). Telecommunication management; Management and orchestration; Management Data Analytics; Requirements and use cases. Technical Specification TS 28.554, 3GPP, 2023. Release 17.

[7] Dario Bega, Mohammad A. Salahuddin, Matteo Gramaglia, Albert Banchs, and Xavier Costa-Pérez. AI-Based Autonomous Control, Management, and Orchestration in 5G: From Standards to Algorithms. Computer Networks, 186:107731, 2021.

[8] 5G PPP Architecture Working Group. View on 5G Architecture Version 4.0. Technical report, 5G Infrastructure Public Private Partnership (5G PPP), July 2021. White Paper, Accessed: 2025-11-05.

[9] Milad Akbari, Raffaele Bolla, Roberto Bruschi, Chiara Lombardo, Nicole Simone Martinelli, and Beatrice Siccardi. A management data analytics function for ethical 6g networks. Computer Networks, 270:111487, 2025.

[10] OpenAI. OpenAI GPT models. https://platform.openai.com/docs/guides/gpt. [Online; Accessed: 2025-11-05].

[11] Google. Gemma 3: Enhanced model for advanced code generation. https://ai.google.dev/gemma/, 2025. Online; Accessed: 2025-11-05.

[12] DeepSeek-AI. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning, 2025.

[13] Minghua He, Tong Jia, Chiming Duan, Huaqian Cai, Ying Li, and Gang Huang. Llmelog: An approach for anomaly detection based on llm-enriched log events. In 2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE), pages 132–143. IEEE, 2024.

[14] Yiheng Zhang, Yunkang Cao, Xiaohao Xu, and Weiming Shen. Logicode: an llm-driven framework for logical anomaly detection. IEEE Transactions on Automation Science and Engineering, 2024.

[15] Mahbod Tavallaee, Ebrahim Bagheri, Wei Lu, and Ali A. Ghorbani. A detailed analysis of the kdd cup 99 data set. Proceedings of the Second IEEE Symposium on Computational Intelligence for Security and Defense Applications, pages 53–58, 2009.

[16] Mohammad Kazim Hooshmand, Manjaiah Doddaghatta Huchaiah, Ahmad Reda Alzighaibi, Hasan Hashim, El-Sayed Atlam, and Ibrahim Gad. Robust network anomaly detection using ensemble learning approach and explainable artificial intelligence (xai). Alexandria Engineering Journal, 94:120–130, 2024.

[17] Nour Moustafa, Jun Yan, and Ali A. Ghorbani. Unsw-nb15: A comprehensive data set for network intrusion detection systems (nids) evaluation. Proceedings of the 2015 Military Communications and Information Systems Conference (MilCIS), pages 1–6, 2015.

[18] Prometheus Authors. Prometheus: Monitoring system and time series database. https://prometheus.io, 2024. Accessed: 2025-11-05.

[19] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. Neural computation, 9(8):1735–1780, 1997.

[20] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pages 1724–1734. Association for Computational Linguistics, 2014.

[21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. Advances in neural information processing systems, 30, 2017.

[22] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In Proceedings of the IEEE International Conference on Computer Vision (ICCV), pages 2980–2988, 2017.

[23] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In Proceedings of the 25th ACM SIGKDD international conference on knowledge discovery & data mining, pages 2623–2631, 2019.

[24] Scott M. Lundberg and Su-In Lee. A Unified Approach to Interpreting Model Predictions, 2017. arXiv preprint arXiv:1705.07874.

[25] Ollama. ollama: Get up and running with large language models locally. https://github.com/ollama/ollama, 2023. Online; Accessed: 2025-11-05.

[26] Meta. Llama 3: Open and efficient foundation models. https://ai.meta.com/blog/meta-llama-3/, 2024. Online; Accessed: 2025-11-05.

[27] OpenAirInterface Software Alliance. Openairinterface: A collaborative open-source software for 4g/5g networks. https://www.openairinterface.org/, 2025. Accessed: 2025-11-06,.

[28] ALİ GÜNGÖR. UERANSIM. https://github.com/aligungr/UERANSIM. [Online; Accessed: 2025-11-05].

[29] Inc. NGINX. Nginx: A high-performance web server and reverse proxy. https://www.nginx.com/, 2025. Accessed: 2025-11-05.

[30] Grafana Labs. Loki: Like Prometheus, but for logs. https://grafana.com/oss/loki, 2024. Accessed: 2025-11-05.

[31] Grafana Labs. Grafana: Open-source platform for monitoring and observability. https://grafana.com/, 2025. Accessed: 2025-11-06,.

[32] InfluxData. InfluxDB: Time-series database platform. https://www.influxdata.com/products/influxdb, 2024. Accessed: 2025-11-05.

Authorized licensed use limited to: Polytechnic University of Bucharest. Downloaded on October 25,2026 at 15:51:38 UTC from IEEE Xplore. Restrictions apply.